**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# LESSON 227 · OOP debugging with IDEs

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. Which structure supports BFS traversal?

Q2. What is an object's field?

Q3. When a local variable shadows a field, which name is used without qualification?

Q4. Why might Step Into be more useful than Step Over for a faulty function?

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# I Do · Read new knowledge

### Key vocabulary

breakpoint — A chosen instruction where execution pauses.

Step Into — Enter a called function for debugging.

Step Over — Execute a call without entering it line by line.

watch — IDE display of chosen variables/expressions.

object state — Current values of an object's fields and properties.

### New learning

Set a breakpoint — Pause at a method call or the following instruction to inspect current state.

Step into to inspect internals — Step Into executes the called method one instruction at a time. Watch Count and parameter n.

Step over to skip internals — Step Over runs the entire call and pauses at the next statement, allowing you to inspect the result.

### Worked example

A Score class has Points=3. Add(5) should change Points to 8. Identify a wrong line.

1. Predict first — Step Into Add and inspect the local points and the field Points.

2. Model the reasoning — The local becomes 8 while the object field remains 3; the assignment targets the wrong variable.

3. Model the reasoning — Replace it with Points += amount; and rerun. The field should be 8.

# We Do · Practise with guidance

A Lamp starts IsOn=false. Toggle() sets IsOn=true on every call. Diagnose the second-call error.

```
void Toggle() {
  IsOn = true; // wrong
}
```

Predict first

After the first call, true is correct; after the second, IsOn should become false.

Model the reasoning

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

The method always assigns true, so the second call fails. Fix to IsOn = !IsOn; and test twice.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Explain Step Into and Step Over at a method call.

### Question 2

A class method Increase(int n) contains `int Total = Total + n;` while Total is an existing property. Identify the issue and write correct C# code.

### Question 3

Design an IDE-debugging workflow for a temperature sensor whose Readings list gains two entries after a single AddReading() call.

## Exit Ticket · Answer before checking

Step Over a method call means it is not executed.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. A FIFO queue.

Q2. Stored state belonging to that object.

Q3. The local variable.

Q4. It lets you inspect statements, parameters and field updates inside the called function.

### You Do pages · Independent questions

Q1. [3] Step Into opens the called method for line-by-line execution [2]; Step Over executes it and stops at the next calling statement [1].

Q2. [5] Local variable shadows property and may be uninitialised/invalid [2]; use `this.Total += n;` [2]; verify by testing initial Total=4, Increase(3) gives 7 [1].

Q3. [6] Set breakpoint before call [1]; step into method [1]; inspect list count before update [1]; step through every append [1]; find duplicate append or duplicate caller invocation [1]; fix and retest one append [1].

### Exit Ticket · Model answer

False. The call executes; the debugger does not stop inside it.

# Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**

**BLACK PEN** Answer all questions first.

**GREEN PEN** Then: tick correct answers; rewrite mistakes.

**Do Now** Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check my understanding